

Getting Help with R

Aside from a google search or visiting [StackOverflow](#), there are some built-in ways to get help with R!

Most R functions have online documentation.

- `help(topic)` documentation on topic ?topic id.
- `help.search("topic")` search the help system
- `apropos("topic")` the names of all objects in the search list matching the regular expression "topic"
- `help.start()` start the HTML version of help
- `str(a)` display the internal *structure* of an R object
- `summary(a)` gives a "summary" of a, usually a statistical summary but it is generic meaning it has different operations for different classes of a
- `ls()` show objects in the search path; specify `pat="pat"` to search on a pattern
- `ls.str()` `str()` for each variable in the search path
- `dir()` show files in the current directory
- `methods(a)` shows S3 methods of a
- `methods(class=class(a))` lists all the methods to handle objects of class a

Let's see some basic examples of how to use these, we won't go through all of them because some of them aren't useful to us...yet!

In [2]:

```
# Need help with vectors, try help()
help(vector)
```

Out[2]:

vector {base}	R Documentation
---------------	-----------------

Vectors

Description

`vector` produces a vector of the given length and mode.

`as.vector`, a generic, attempts to coerce its argument into a vector of mode `mode` (the default is to coerce to whichever vector mode is most convenient): if the result is atomic all attributes are removed.

`is.vector` returns `TRUE` if `x` is a vector of the specified mode having no attributes *other than names*. It returns `FALSE` otherwise.

Usage

```
vector(mode = "logical", length = 0)
as.vector(x, mode = "any")
is.vector(x, mode = "any")
```

Arguments

mode	character string naming an atomic mode or "list" or "expression" or (except for vector) "any".
length	a non-negative integer specifying the desired length. For a long vector, i.e., <code>length > .Machine\$integer.max</code> , it has to be of type "double". Supplying an argument of length other than one is an error.
x	an R object.

Details

The atomic modes are "logical", "integer", "numeric" (synonym "double"), "complex", "character" and "raw".

If `mode = "any"`, `is.vector` may return `TRUE` for the atomic modes, list and expression. For any mode, it will return `FALSE` if `x` has any attributes except names. (This is incompatible with S.) On the other hand, `as.vector` removes *all* attributes including names for results of atomic mode (but not those of mode "list" nor "expression").

Note that factors are *not* vectors; `is.vector` returns `FALSE` and `as.vector` converts a factor to a character vector for `mode = "any"`.

Value

For `vector`, a vector of the given length and mode. Logical vector elements are initialized to `FALSE`, numeric vector elements to 0, character vector elements to "", raw vector elements to nul bytes and list/expression elements to `NULL`.

For `as.vector`, a vector (atomic or of type list or expression). All attributes are removed from the result if it is of an atomic mode, but not in general for a list result. The default method handles 24 input types and 12 values of `type`: the details of most coercions are undocumented and subject to change.

For `is.vector`, `TRUE` or `FALSE`. `is.vector(x, mode = "numeric")` can be true for vectors of types "integer" or "double" whereas `is.vector(x, mode = "double")` can only be true for those of type "double".

Methods for `as.vector()`

Writers of methods for `as.vector` need to take care to follow the conventions of the default method. In particular

- Argument `mode` can be "any", any of the atomic modes, "list", "expression", "symbol", "pairlist" or one of the aliases "double" and "name".
- The return value should be of the appropriate mode. For `mode = "any"` this means an atomic vector or list.
- Attributes should be treated appropriately: in particular when the result is an atomic vector there should be no attributes, not even names.
- `is.vector(as.vector(x, m), m)` should be true for any mode `m`, including the default "any".

Note

`as.vector` and `is.vector` are quite distinct from the meaning of the formal class "vector" in the methods package, and hence `as(x, "vector")` and `is(x, "vector")`.

Note that `as.vector(x)` is not necessarily a null operation if `is.vector(x)` is true: any names will be removed from an atomic vector.

Non-vector modes "symbol" (synonym "name") and "pairlist" are accepted but have long been undocumented: they are used to implement `as.name` and `as.pairlist`, and those functions should preferably be used directly. None of the description here applies to those modes: see the help for the preferred forms.

References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

See Also

`c`, `is.numeric`, `is.list`, etc.

Examples

```
df <- data.frame(x = 1:3, y = 5:7)
## Error:
try(as.vector(data.frame(x = 1:3, y = 5:7), mode = "numeric"))
```

```
x <- c(a = 1, b = 2)
is.vector(x)
as.vector(x)
all.equal(x, as.vector(x)) ## FALSE
```

```
###-- All the following are TRUE:
is.list(df)
! is.vector(df)
! is.vector(df, mode = "list")
```

```
is.vector(list(), mode = "list")
```

Very nice! We can also do a search:

In [4]:

```
# This will pop up a help window (need to pass a character string)
help.search('numeric')
```

In [5]:

```
# Can also use ?? for a search
??vector
```

In [6]:

```
# Can also do a quick stats summary:
v <- c(1,2,3,4,5,6)
summary(v)
```

Out[6]:

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.00	2.25	3.50	3.50	4.75	6.00

We will use these help tools later on when we want more information, for now just keep **help()** and **??** in mind when your stuck on something (as well as google searching!).

That's all for now!